



ДИСЦИПЛИНА

Разработка приложений на языке Котлин

(полное наименование дисциплины без сокращений)

ИНСТИТУТ

информационных технологий

КАФЕДРА

информационных технологий в атомной энергетике

полное наименование кафедры

ВИД УЧЕБНОГО
МАТЕРИАЛА

Лекция

(в соответствии с пп 1-11)

ПРЕПОДАВАТЕЛЬ

Золотухин Святослав Александрович

(фамилия, имя, отчество)

СЕМЕСТР

5 семестр 2025 – 2026 учебный год

(указать семестр обучения, учебный год)



Лекция 6. Разработка приложений на языке Котлин

ФИО преподавателя: Золотухин Святослав Александрович

e-mail: zolotuhin@mirea.ru



Обобщения (Generics)

Обобщенные классы и функции

```
class Car<T>(val id: T, val title: String)  
fun main() {  
    val car1: Car<Int> = Car(1, "Mercedes")  
    val car2: Car<String> = Car("17A", "Lada")  
  
    println("${car1.id} - ${car1.name}")  
    println("${car2.id} - ${car2.name}")  
}
```



Обобщения (Generics)

```
class Car<T>(val id: T, val title: String){  
    fun checkID(_id: T){  
        if(id == _id){  
            println(" Такой же")  
        }  
        else{  
            println("Другой")  
        }  
    }  
}
```



Обобщения (Generics)

```
fun main() {  
    val car1 = Car("ID1", "Car")  
    car1.checkID("ID1 ") // The same  
    car1.checkID("ID323") // Different  
}
```



Обобщения (Generics)

```
val humans: Array<String> = arrayOf("Andrew", "Kate", "Vasya")
```

```
val nums: Array<Int> = arrayOf(1, 2, 3, 4, 5)
```

Array<T>. Параметр класса определяет, элементы какого типа массив будет хранить



Обобщения (Generics)

Применение нескольких параметров:

```
fun main() {  
    val translation1: Translation<String, String> = Translation("apple",  
        "яблоко")  
    val translation2: Translation<String, Int> = Translation("banana", 2)  
  
    println("${translation1.source} - ${translation1.target}") // apple - яблоко  
    println("${translation2.source} - ${translation2.target}") // banana - 2  
}  
  
class Translation<K, V>(val source: K, var target: V)
```



Обобщения (Generics)

Обобщенные функции:

```
fun main() {  
    smth("Words")  
    smth(123)  
    smth(true)  
}  
  
fun <T> smth(obj: T){  
    println(obj)  
}
```



Обобщения (Generics)

```
fun main() {  
    val numbers1 = findLarger(arrayOf(10, 20, 30, 40), arrayOf(30, 40, 50, 60, 70, 70))  
    numbers1.forEach { item -> print("$item ") } // 30 40 50 60 70 70  
  
    println()  
    val names1 = findLarger(arrayOf("John", "Mike", "Alice"), arrayOf("Eva", "Olivia"))  
    names1.forEach { item -> print("$item ") } // John Mike Alice  
}  
  
fun <T> findLarger(collection1: Array<T>, collection2: Array<T>): Array<T> {  
    return if (collection1.size > collection2.size) collection1 else collection2  
}
```



Ограничение обобщений

```
fun <T> getBiggest(a: T, b: T): T{  
    if(a > b) return a  // ! Ошибка  
    else return b  
}
```



Ограничение обобщений

```
fun main() {  
    val res1 = getBiggest(1, 2)  
    println(res1)  
    val res2 = getBiggest("Tatiana", "Sergey")  
    println(res2)  
}  
  
fun <T: Comparable<T>> getBiggest(a: T, b: T): T {  
    return if(a > b) a  
    else b  
}
```



Ограничение обобщений

```
fun<T:Message> send(message: T){  
    println(message.text)  
}
```

```
interface Message{  
    val text: String  
}
```

```
class EmailMessage(override val text: String): Message  
class SmsMessage(override val text: String): Message
```



Ограничение обобщений

```
fun main() {  
    val email1 = EmailMessage("Hello")  
    send(email1)  
    val sms1 = SmsMessage("Привет")  
    send(sms1)  
}
```



Ограничение обобщений

Установка нескольких ограничений

```
fun <T> getBiggest(a: T, b: T): T where T: Comparable<T>,  
                                     T: Number {  
    return if(a > b) a  
    else b  
}
```



Ограничение обобщений

Установка нескольких ограничений

```
fun main() {  
    val res1 = getBiggest(1, 2)  
    println(res1) // 2  
  
    val res2 = getBiggest(1.6, -2.8)  
    println(res2) // 1.6  
  
    // val res3 = getBiggest("Tatiana", "Sergey") // ! Ошибка -  
    String не является производным от класса Number  
    // println(res3)
```



Null и nullable-типы

Kotlin призван исключить ошибки рода `NullPointerException` из нашего кода. NPE могут возникать только в случае:

- Явного указания `throw NullPointerException();`
- Использования оператора `!!` (далее);
- Эту ошибку вызвал внешний Java-код;
- Есть какое-то несоответствие при инициализации данных (в конструкторе использована ссылка `this` на данные, которые не были ещё проинициализированы).



Null и nullable-типы

Типы, которые допускают значение null, должны быть помечены вопросительным знаком

var k:Int = 0 //null - недопустимое значение k

var k1:Int? = null //null может быть значением k1

Переменная часто используемого типа String не может быть null:

var a: String = "abc"

a = null // ошибка компиляции



Null и nullable-типы

Чтобы разрешить null значение, мы можем объявить эту строковую переменную как «String?»:

```
var b: String? = "abc"
```

```
b = null // ok
```

Теперь, при вызове метода с использованием переменной a, исключены какие-либо NPE. Вы спокойно можете писать:

```
val x = a.length
```

В случае, если вы захотите получить доступ к значению b, это будет небезопасно:

```
val x = b.length // ошибка: переменная b может быть null
```



Null и nullable-типы

1 способ. Проверка на null.

val l = if (b != null) b.length else -1 // явно проверить b на null значение и обработать два варианта по отдельности

```
if (b != null && b.length > 0) {  
    print("String of length ${b.length}")  
} else {  
    print("Empty string")  
}
```



Null и nullable-типы

2 способ. Оператор безопасного вызова (?.)

**`println(b?.length)` // код возвращает `b.length` в том, случае, если `b` не имеет значение `null`. Иначе он возвращает `null`.
Типом этого выражения будет `Int?`**



Null и nullable-типы

Оператор объединения по null (?:)

```
val name : String? = "Tom"
```

```
val length: Int? = name?.length
```

```
val length: Int?
```

```
if(name != null)
```

```
    length = name.length
```

```
else
```

```
    length = null
```



Null и nullable-типы

```
val name : String? = "Timon"
```

```
val length: Int = name?.length ?:0
```

Теперь переменная `length` не допускает значения `null`. И если переменная `name` не определена, то `length` получает число 0. Оператор `?.` обеспечивает саму возможность вызова свойства `length`, а оператор `?:` определяет какое значение должно присвоиться в `length`, если `name = null`



Null и nullable-типы

Оператор контроля non-null (!!)

```
val company : String? = "Samsung"
```

```
val id: String = company!!
```

```
println(id)
```

```
val company : String? = null
```

```
val length :Int = company!!.length
```



Преобразование типов

Встроенные методы преобразования типов:

- **toByte**
- **toShort**
- **toInt**
- **toLong**
- **toFloat**
- **toDouble**
- **toChar**



Преобразование типов

```
val s: String = "12"
```

```
val d: Int = s.toInt()
```

```
println(d)
```

```
val d: Int = "12" // ! Ошибка
```

```
val s: String = "tom"
```

```
try {
```

```
    val d: Int = s.toInt()
```

```
    println(d)
```

```
}
```

```
catch(e: Exception){
```

```
    println(e.message)
```



Преобразование типов

Smart Cast и оператор is

Оператор is:

- Оператор is используется для проверки типа переменной.
- Если переменная соответствует указанному типу, условие истинно.
- После проверки с использованием is, Kotlin автоматически приведет тип переменной внутри блока условия.

```
val obj: Any = "Hello"
```

```
if (obj is String) {
```

```
    // obj автоматически приведен к типу String внутри этого блока
```

```
    println(obj.length)
```

```
}
```



Преобразование типов

Smart Cast:

- Smart Cast представляет собой автоматическое приведение типов после проверки с использованием `is`.
- Kotlin "помнит" результат проверки типа, и в последующем коде допускает использование переменной с типом, который был проверен.

```
val obj: Any = "Hello"
```

```
if (obj is String) {
```

```
    // obj автоматически приведен к типу String
```

```
    println(obj.length)
```

```
    // Допустимо использовать obj как String без явного приведения
```

```
    val upperCase: String = obj.toUpperCase()
```

```
}
```



Функции расширения

Функции расширения (extension function) позволяют добавить функционал к уже определенным типам. При этом типы могут быть определены где-то в другом месте, например, в стандартной библиотеке.

Функция расширения определяется следующим образом:

```
fun тип.имя_функции(параметры) : возвращаемый_тип{  
    тело функции  
}
```



Функции расширения

```
fun main() {  
    val hello: String = "hello world"  
    println(hello.wordCount('l')) // 3  
    println(hello.wordCount('o')) // 2  
    println(4.square())           // 16  
    println(6.square())           // 36  
}
```

```
fun String.wordCount(c: Char) : Int{  
    var count = 0  
    for(n in this){  
        if(n == c) count++  
    }  
    return count  
}  
fun Int.square(): Int{  
    return this * this  
}
```



Score-функции

В Kotlin есть пять подобных функций: **let**, **run**, **with**, **apply** и **also**. Эти функции похожи по своему действию и различаются прежде всего по параметрам и возвращаемым результатам. Функция **let**:

```
val name: String? = "John"
```

```
// Пример без использования let
```

```
if (name != null) {  
    val length = name.length  
    println("Name length is $length")  
}
```

```
// Тот же пример, используя let
```

```
name?.let {  
    val length = it.length  
    println("Name length is $length")  
}
```



Score-функции

Функция – расширения **with**

Функция **with** в Kotlin предоставляет удобный способ работать с объектом, неявно обращаясь к его членам. Она позволяет вызывать методы и обращаться к свойствам объекта, не повторяя имя объекта каждый раз.

```
fun main() {  
    val person = Person()  
  
    with(person) {  
        name = "John"  
        age = 30  
    }  
    println("Name: ${person.name}, Age: ${person.age}")  
}
```

```
class Person {  
    var name: String = ""  
    var age: Int = 0  
}
```



Score-функции

Run

Функция `run` в Kotlin также предоставляет удобный способ работы с объектами, но в отличие от `with`, она является расширением для любого объекта и может использоваться без создания блока контекста.

```
fun main() {  
    val person = Person().run {  
        name = "Alice"  
        age = 25  
        this // возвращаем объект Person  
    }  
  
    println("Name: ${person.name}, Age: ${person.age}")  
}
```

```
class Person {  
    var name: String = ""  
    var age: Int = 0  
}
```



Score-функции

Функция **apply** в Kotlin также предоставляет удобный способ работы с объектами и их инициализацию. Эта функция является частью стандартной библиотеки и используется как расширение для любого объекта. Она возвращает сам объект после выполнения блока кода

```
fun main() {  
    val person = Person().apply {  
        name = "Bob"  
        age = 30  
    }  
    println("Name: ${person.name}, Age: ${person.age}")  
}
```

```
class Person {  
    var name: String = ""  
    var age: Int = 0  
}
```



Score-функции

Функция **also** также предоставляет удобный способ работы с объектами, подобно другим функциям, таким как **apply**, **let**, и т. д. Однако **also** возвращает сам объект, а не результат выполнения блока кода. Это позволяет проводить дополнительные действия с объектом в процессе его использования

```
fun main() {  
    val person = Person("Alice", 25).also {  
        it.age += 1  
        println("Updating age in 'also' block")  
    }  
    println("Name: ${person.name}, Updated Age: ${person.age}")  
}
```

```
data class Person(var  
    name: String, var age: Int)
```



Спасибо за внимание!